FulfillOps: Resilient Asynchronous Order-Processing Platform

Final project report

Module: DevOps Approach

Project: FulfillOps

Author: lil mans (Jakub MOTYSIA , Mohith AYANURU)

Report purpose: Technical overview, implementation evidence, evaluation, and limitations of the project

---

1. Executive summary

FulfillOps is a resilient order-processing platform for a fictional electronics fulfillment business. The system accepts customer orders through a FastAPI HTTP service, reserves inventory atomically in PostgreSQL, and processes accepted orders asynchronously through a separate worker process. The design addresses the failure modes that make order systems difficult to operate: client retries, duplicate requests, simultaneous purchases of limited stock, process restarts, worker crashes, and permanently failing jobs.

The central design decision is to keep authoritative state in PostgreSQL rather than in API or worker memory. A single transaction creates the order, reserves the requested stock, records the idempotency mapping, and creates a durable work item. This means that an HTTP response is returned only after the accepted order can survive an API restart. The worker later claims the durable work item, transitions the order, retries failures with bounded exponential backoff, and retains exhausted jobs in a dead-letter state for operator review.

The API follows the supplied version 1.1 contract. It exposes liveness and readiness checks, catalog and inventory queries, order creation and lookup, and Prometheus-format metrics. The implementation also handles duplicate SKU lines, strict validation, request-size limits, safe error responses, and idempotent replay. The documented validation and reliability checks include local tests, concurrent public-checker runs, a poison-order demonstration, and CI evidence for a failed security scan followed by a corrected green run.

The solution was also deployed to a local Kubernetes cluster and reconciled by Argo CD. The operational evidence includes a synced and healthy application, drift detection, a deliberately broken rollout, a rolling restart probed for 30 seconds with no failed readiness requests, a worker-pod deletion drill, and a 200-sample latency measurement. This report describes both the technical solution and that evidence. Where evidence is missing or only partly captured, it is identified as a limitation in section 12 rather than presented as completed.

2. Problem statement and objectives

An order platform must do more than return a successful HTTP status. It must protect the consistency of stock and order state when requests arrive concurrently and when infrastructure fails. FulfillOps was designed around the following practical scenario:

1. A customer submits an order containing one or more products.
2. The API validates the request and reserves inventory immediately.
3. The customer receives an acceptance receipt without waiting for all downstream processing to finish.
4. A separate worker completes the order asynchronously.
5. A retry of the same request must not create another order or reserve stock twice.
6. A different order must not obtain stock that has already been committed.
7. A worker crash must not make a committed order disappear.
8. A permanently failing order must become visible to an operator instead of retrying forever.

The project objectives were therefore:

- implement the required HTTP contract exactly;
- establish clear data and state invariants;
- make inventory reservation atomic under concurrency;
- provide durable at-least-once work delivery with an exactly-once business effect;
- validate the design with unit, integration, concurrency, and black-box tests;
- package the services for reproducible local execution;
- apply security and supply-chain safeguards; and
- document decisions, evidence, limitations, and individual contribution.

The authoritative API details are defined in `API_CONTRACT_EN.md`, while the architecture decision is recorded in `ADR-001.md`.

3. Requirements and success criteria

The system has three groups of requirements.

3.1 Functional requirements

The catalog contains three seeded products:

SKU	Unit price	Seed quantity
`KIT-100`	1,299 cents	7
`PART-200`	599 cents	12
`PACK-300`	249 cents	5

An order contains a customer identifier and one to ten item lines. Quantities must be positive integers, SKUs must be known, and a client must provide an `Idempotency-Key`. The order total is calculated from the persisted catalog prices. Inventory is decremented when the order is accepted, not when the worker later completes it.

3.2 Reliability requirements

The important invariants are:

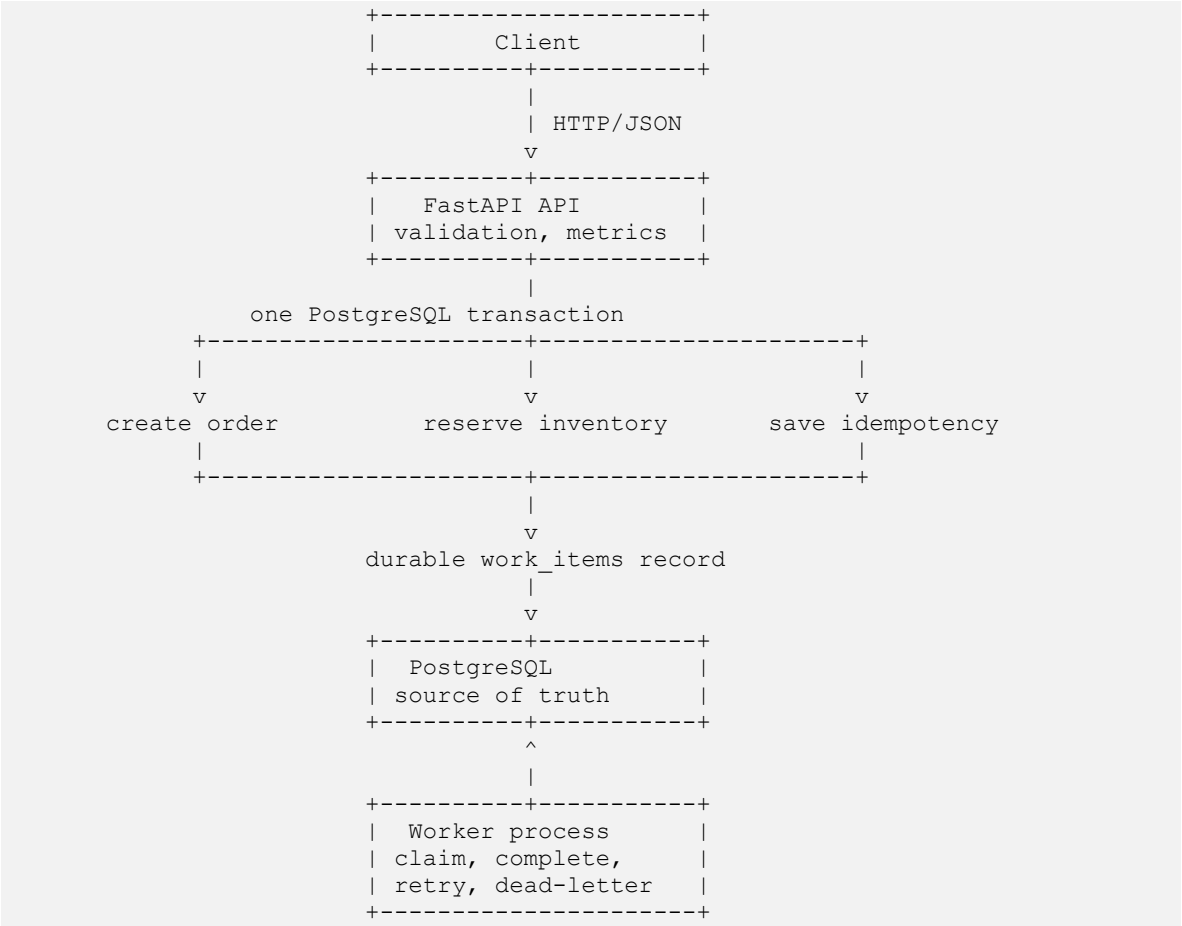
- `inventory.available` is never negative;
- one idempotency key maps to at most one order;
- identical replays return the original order identifier;
- a changed payload using an existing key is rejected;
- duplicate SKU lines are aggregated before stock validation;
- an accepted order and its work item are committed together; and
- a worker retry cannot apply the completion effect twice.

3.3 Operational requirements

The project brief also calls for reproducible containers, CI quality gates, security scanning, a software bill of materials, health probes, metrics, controlled failure demonstrations, and a Kubernetes/GitOps deployment. The available documentation contains the design, CI evidence, and Kubernetes/GitOps, rollout and latency evidence captured on a local cluster. The report distinguishes requirements that have captured evidence from those that remain incomplete.

4. Architecture and design decisions

4.1 Logical architecture



The API and worker are separate processes. PostgreSQL stores products, inventory, orders, order items, idempotency mappings, and durable work items. This keeps business state independent of the lifetime of any individual container. The database is not merely a cache or a queue backup; it is the authoritative source for both order state and inventory.

4.2 Why PostgreSQL was selected

PostgreSQL provides the transaction and locking primitives required by the problem without introducing a second mandatory stateful service. The same database can atomically persist the order and its work item, lock inventory rows, enforce uniqueness, and recover work after a worker process terminates. It is also straightforward to reproduce locally with Docker Compose and can support multiple API and worker replicas.

The trade-off is that the worker state machine must be implemented carefully. At-least-once delivery means a job may be observed more than once after a lease expires, so the order transition and completion effect must be guarded by persisted state. The design accepts this complexity in exchange for a small, durable architecture that is easy to inspect and test.

4.3 Data model

The principal entities are:

- **products:** SKU and unit price;
- **inventory:** current available quantity per SKU;
- **orders:** customer, status, total, and timestamps;
- **order_items:** the normalized product quantities belonging to an order;
- **idempotency_keys:** normalized request hash and the associated order ID; and
- **work_items:** durable processing state, attempts, lease information, backoff time, and the last error.

The schema migration and seed data provide a repeatable starting point. A unique idempotency constraint prevents two committed mappings for the same client key. Foreign keys preserve order/item and item/product relationships. The work table retains failed records rather than silently deleting them.

5. API implementation

The service implements the required routes:

Endpoint	Purpose
`GET /health/live`	process liveness, independent of the database
`GET /health/ready`	database and migration readiness
`GET /api/v1/catalog`	seeded products and prices
`GET /api/v1/inventory/{sku}`	current non-negative availability
`POST /api/v1/orders`	validate, reserve, persist, and queue an order
`GET /api/v1/orders/{order_id}`	retrieve order status and total
`GET /metrics`	Prometheus-format operational metrics

The service uses machine-readable error codes. Examples include `INVALID_JSON`, `INVALID_REQUEST`, `INVALID_QUANTITY`, `INVALID_SKU`, `OUT_OF_STOCK`, `IDEMPOTENCY_REQUIRED`, `IDEMPOTENCY_CONFLICT`, and `ORDER_NOT_FOUND`. Error messages are non-empty and safe: they do not reveal SQL statements, credentials, stack traces, or connection strings.

Input handling is intentionally strict. The API rejects missing or malformed idempotency keys, empty customer identifiers, invalid item structures, non-positive quantities, unknown SKUs, and requests larger than 16 KiB. Boolean values are not accepted as integer quantities.

These checks reduce ambiguous behavior and prevent an invalid request from reaching the reservation transaction.

5.1 Order acceptance flow

For a new order, the application performs the following sequence:

1. Parse and validate the request.
2. Normalize the payload and calculate a stable request hash.
3. Acquire a PostgreSQL transaction advisory lock for the idempotency key.
4. Look up an existing mapping.
5. Return the original order for an identical replay, or reject a changed payload with `409 IDEMPOTENCY_CONFLICT`.
6. Aggregate repeated SKU lines.
7. Lock the relevant inventory rows with `SELECT ... FOR UPDATE`.
8. Check all requested quantities before making any change.
9. Decrement inventory.
10. Create the order and normalized order items.
11. Insert the idempotency mapping.
12. Insert a durable work item.
13. Commit the transaction.

The `202 Accepted` response is sent only after step 13. Consequently, a successful response means that the order, reservation, replay mapping, and future work are all durable. If any step fails, the transaction rolls back and there is no partial reservation.

5.2 Duplicate SKU aggregation

A request can contain the same SKU on multiple lines. The service combines those lines before checking availability and calculating the order total. For example, two lines for `KIT-100` with quantities four and four become one effective request for eight units. Since the seed inventory is seven, the whole order is rejected as `OUT_OF_STOCK`; four units are not reserved partially. This behavior is important because validating each line independently would make the final result dependent on line ordering.

6. Worker processing and failure handling

The worker continuously searches for pending or recoverable work. It claims a job inside a database transaction and records a lease. PostgreSQL `FOR UPDATE SKIP LOCKED` allows multiple workers to make progress without waiting on jobs already claimed by another worker.

The work-item state machine is conceptually:

```
pending -> processing -> completed
    ^          |
    |          v
    +------ retryable failure
                  |
                  v
             dead_letter
```

When a job is claimed, the associated order moves to `processing`. If the worker completes successfully, the order moves to `completed` and the work item is finalized. If the worker fails, the attempt count increases and the next execution time is calculated using bounded exponential backoff. Once the maximum attempt count is reached, the work item becomes `dead_letter` and the order becomes `failed`.

The claim timeout is essential for crash recovery. If a worker terminates after claiming a job but before completing it, the lease eventually becomes stale. Another worker can then return the item to `pending` and process it. Because the business transition is guarded by persisted order state, recovery does not complete the same order twice.

The deterministic `POISON-ORDER` hook is used only for testing. It produces a repeatable failure, demonstrates bounded retries, and leaves the failed work available for operator review. The design deliberately does not automatically restock inventory after a permanent failure: that decision requires business reconciliation and should not be guessed by infrastructure code.

7. Testing strategy and results

The testing approach is layered so that a passing black-box test is supported by smaller, diagnosable checks.

7.1 Test layers

Layer	Main purpose
Unit	validation, normalization, aggregation, totals, retry policy
Integration	order acceptance, persistence, worker completion, replay
Race	same-key concurrency and competing-key stock contention
Worker	claim recovery, backoff, poison jobs, no double completion
Black-box	supplied public HTTP contract checker
Operational	rollout, fault injection, metrics, and recovery evidence

All destructive tests are intended for an isolated test database. The reset tooling checks the application environment and database naming before recreating test data. This prevents a test convenience from becoming a production data-loss mechanism.

7.2 Local checks

The Partner 2 integration branch records seventeen passing pytest tests. These cover liveness, invalid JSON, required idempotency keys, invalid quantities, unknown orders, request-size protection, same-key and competing-key races, stale-claim behavior, retry backoff, poison jobs, two-worker claim safety, and worker crash recovery. This is stronger evidence than the earlier nine-test baseline because it includes the independently owned race and worker reliability tests.

The test plan also defines the two most important race scenarios:

1. Eight simultaneous requests with the same idempotency key across two API instances must return one stable order ID and decrement stock once.
2. Twelve distinct-key requests competing for five `PACK-300` units must accept exactly five and reject seven with `OUT_OF_STOCK`, without negative inventory.

Other important cases include changed payloads under an existing key, reordered JSON keys, replay after an API restart, duplicate SKU aggregation, worker crash recovery, poison-job termination, and increasing retry gaps.

The recorded Partner 2 branch results are:

```
2 cross-replica race tests passed
17 local pytest tests passed
```

The cross-replica tests run eight identical-key requests and twelve different-key requests against two API services backed by the same PostgreSQL database. The worker reliability suite verifies that two workers do not claim the same job, that a job abandoned by a killed worker is reclaimed and completed once, and that retry delay remains bounded.

7.3 Public contract checker

The public checker was run against a disposable Docker Compose stack with concurrency and stress enabled:

```
python fulfillops_public_checker.py `
  --base-url http://localhost:8080 `
  --concurrency `
  --stress
```

Recorded result:

```
15 passed / 0 failed / 0 skipped
```

The result covers catalog behavior, request validation, idempotent replay, worker completion, exact inventory reservation, concurrent reservations, same-key concurrency, duplicate SKU aggregation, and metrics exposure. Since the checker consumes inventory, the run must always use a freshly seeded isolated test database.

7.4 Poison-job result

The isolated poison-order demonstration produced:

```
order status: failed
work item: dead_letter
attempts: 3
last error: Deterministic poison order
```

This is evidence that the worker stops after a bounded number of attempts and retains the failure instead of deleting it or entering an infinite retry loop.

8. DevOps, containerization, and delivery

8.1 Local reproducibility

The intended local workflow is:

```
docker compose up -d --build
docker compose ps
curl http://localhost:8080/health/ready
python -m pytest
```

The API, worker, and PostgreSQL services are separated so that the worker can be restarted independently of the HTTP process. PostgreSQL uses persistent storage. The runtime should be rebuilt from explicit dependency manifests, with environment variables supplying credentials rather than committing secrets.

The container hardening requirements include a non-root runtime user, minimal runtime dependencies, a `.dockerignore`, explicit versions, and health checks. The safe reset command is restricted to the disposable test environment.

8.2 CI quality gates

The documented CI pipeline contains linting, tests, contract checks, Docker image build, vulnerability scanning, and SBOM generation. The pipeline evidence demonstrates a useful red-to-green quality gate:

- the initial run failed because Trivy found five fixable HIGH findings;
- the affected OS and Python dependencies were upgraded;
- the Docker image OS packages were patched; and
- the corrected run passed lint/tests/checker and image/scan/SBOM jobs.

The documented green run reports the lint/tests/checker and image/scan/SBOM jobs as successful. The branch-local evidence records seventeen pytest tests and a successful concurrency/stress checker run. The SBOM is generated in SPDX JSON format using Syft tooling and is retained as a CI artifact. This makes the dependency inventory inspectable rather than relying only on a successful build.

The project also records a supply-chain lesson: action references must be reviewed and pinned to safe, existing releases. A non-existent or compromised action tag can undermine an otherwise correct pipeline.

8.3 Kubernetes and GitOps deployment

Beyond Docker Compose, the solution was deployed to a local Kubernetes cluster (kind, cluster kind-fulfillops, namespace fulfillops). The manifests are stored in the repository under `deploy/k8s` and are reconciled by Argo CD v3.6.1, so the Git repository is the declared source of truth for the running system.

Item	Captured value
Argo CD version	v3.6.1
Repository / path	github.com/Jakub-Tenders/FULFILLOPS, path `deploy/k8s`
Destination	in-cluster, namespace `fulfillops`
Target revision in the screenshots	`feature/k8s` (commit `c5f0b2e`)
Sync policy	Auto sync enabled
Application status	Healthy and Synced
Workloads	2 API pods, 2 worker pods, 1 PostgreSQL pod, all Running 1/1

The Argo CD resource tree shows the application as Healthy and Synced, with nine resources Synced and none OutOfSync. The managed resources include the configuration and database-initialisation ConfigMaps, the namespace, the API service and a NodePort service, the PostgreSQL service, the `fulfillops-api` and `fulfillops-worker` deployments, and the `postgres` StatefulSet with its persistent volume claim `data-postgres-0`. The persistent volume claim is the evidence for persistent database storage in the cluster.

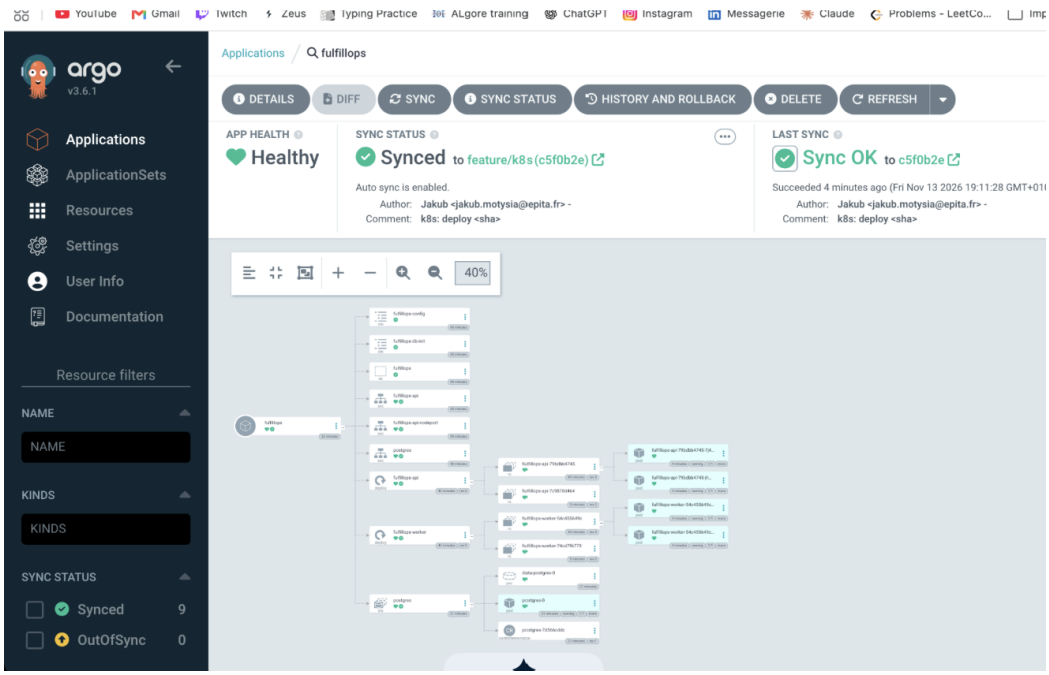


Figure 1: *Argo CD application tree for fulfillops: Healthy, Synced to feature/k8s (c5f0b2e), auto sync enabled, 9 resources Synced and 0 OutOfSync.*

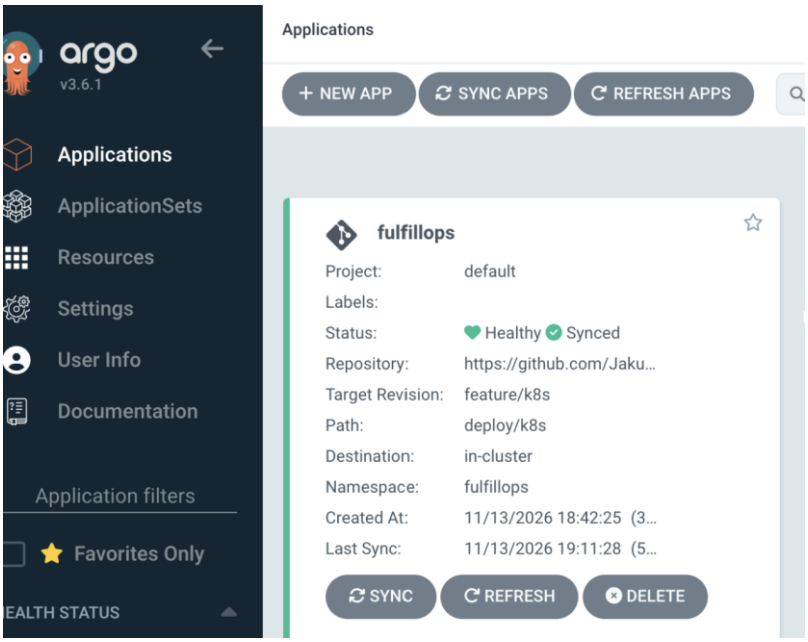


Figure 2: *Argo CD application card: project default, repository, target revision feature/k8s, path deploy/k8s, namespace fulfillops, Healthy and Synced.*

```
(venv) macbook-air-de-jakub:FulFillops jakubmotysia$ git push origin feature/k8s
(venv) macbook-air-de-jakub:FulFillops jakubmotysia$ kubectl -n fulfillops get pods
NAME                                  READY   STATUS    RESTARTS   AGE
fulfillops-api-79bdbb4745-7j4dr       1/1     Running   0          56m
fulfillops-api-79bdbb4745-jhm4j       1/1     Running   0          57m
fulfillops-worker-54c455649c-5clxw    1/1     Running   0          57m
fulfillops-worker-54c455649c-8d5zc    1/1     Running   0          57m
postgres-0                            1/1     Running   0          74m
(venv) macbook-air-de-jakub:FulFillops jakubmotysia$
```

Figure 3: *kubectl get pods in namespace fulfillops: two API pods, two worker pods and postgres-0, all Running 1/1 with 0 restarts.*

The API manifest (`deploy/k8s/api.yaml`) runs the container as a non-root user and group (10001) with the RuntimeDefault seccomp profile. It reads configuration from the `fulfillops-config` ConfigMap, takes the database URL from a Kubernetes Secret reference, and defines readiness and liveness probes (liveness on `/health/live` with a 10-second initial delay and period). A `preStop` hook sleeps for five seconds before the container stops. These settings support the container-hardening and health-check requirements described earlier.

8.4 Drift detection and self-healing

To test GitOps reconciliation, the live `fulfillops-config` ConfigMap was edited directly in the cluster through the Argo CD interface, changing `WORKER_POLL_INTERVAL_SECONDS` from the declared value of 1 to 2. Argo CD flagged the resource as OutOfSync because the live state no longer matched the manifest in Git. Afterwards the application returned to Synced and Healthy with nine resources Synced and none OutOfSync, and the last-sync time moved from 19:11:28 to 19:26:30 (Argo CD displays local time, GMT+01:00), which is consistent with automated reconciliation back to the declared state.

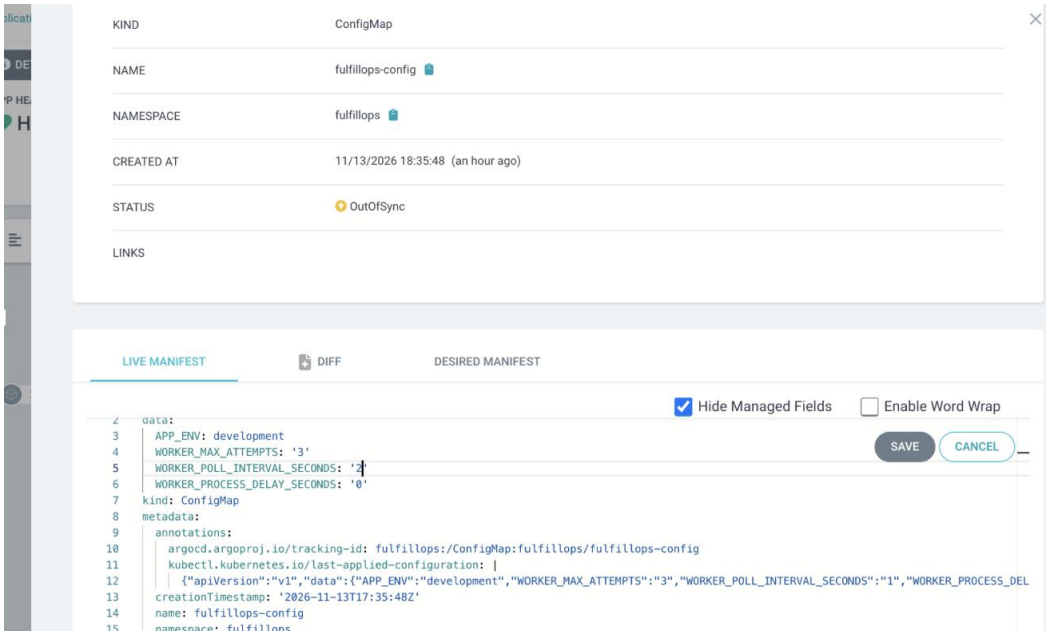


Figure 4: *Live manifest of ConfigMap fulfillops-config being edited in Argo CD (WORKER_POLL_INTERVAL_SECONDS typed as 2 over the declared 1); the resource status is OutOfSync.*

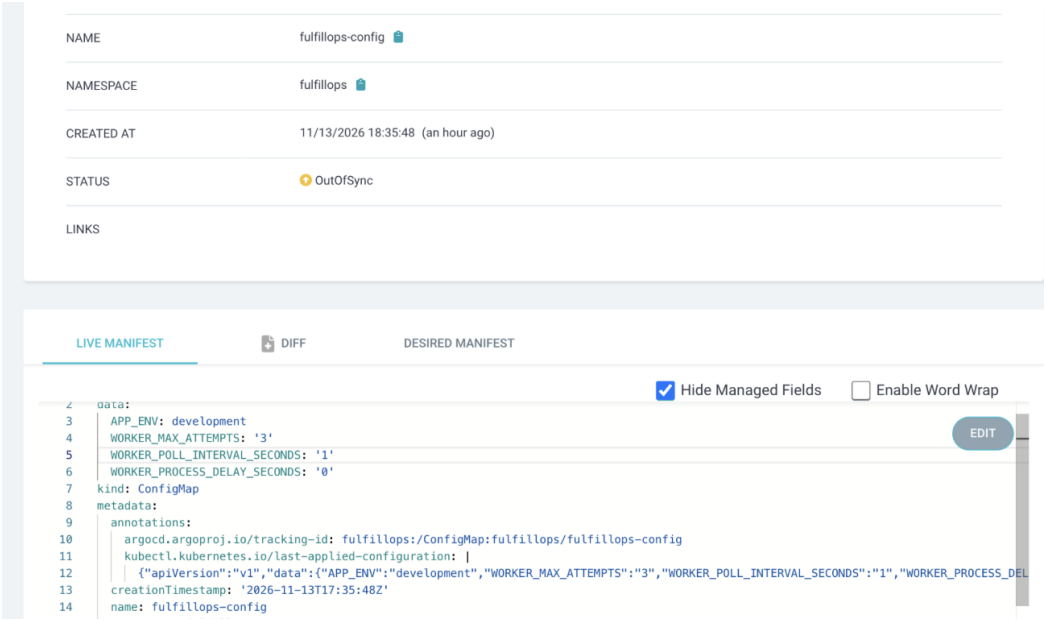


Figure 5: *The same ConfigMap with status OutOfSync; the declared value of WORKER_POLL_INTERVAL_SECONDS in the live manifest is 1.*

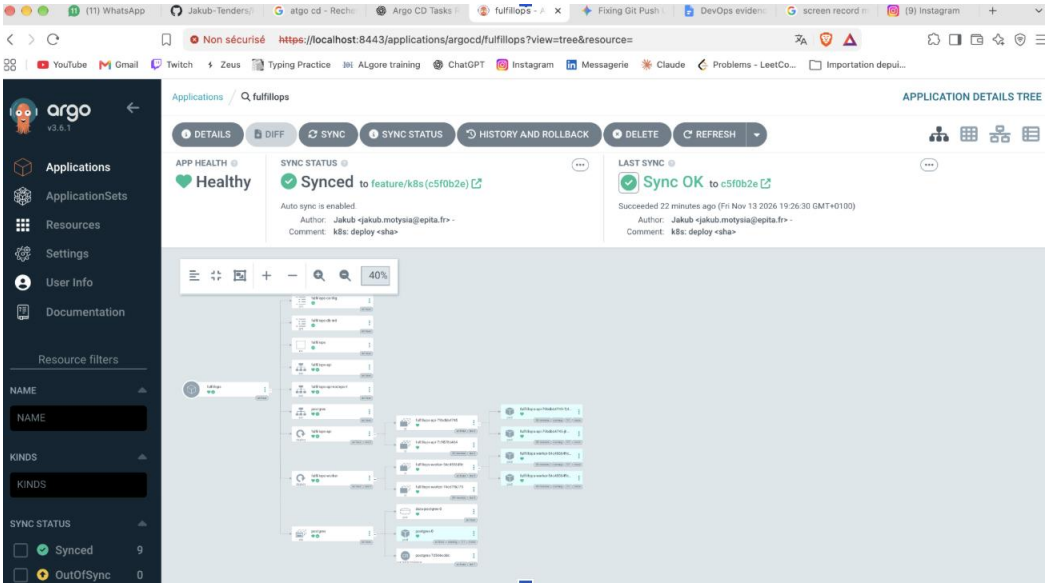


Figure 6: *After reconciliation: application Healthy and Synced, 9 Synced and 0 OutOfSync, last sync 19:26:30.*

The project notes also describe a second drift test in which the API deployment was scaled from two replicas to one with `kubectl` and was restored to two by Argo CD. That run is recorded in the notes, but no screenshot of it is included in this report.

8.5 Broken rollout

To test a bad release, the readiness probe path in `deploy/k8s/api.yaml` was changed to `/health/does-not-exist` and the change was committed (`cfae581`, message “test: demonstrate broken Kubernetes rollout”) and pushed. Argo CD synced the commit and reported the application as Degraded, with the cause identified as the `fulfillops-api` deployment. The resource tree shows the new ReplicaSet’s pod not yet healthy while the

earlier API pods remain healthy. A rolling update only removes old pods once new ones pass readiness, so the faulty release did not take the service down.

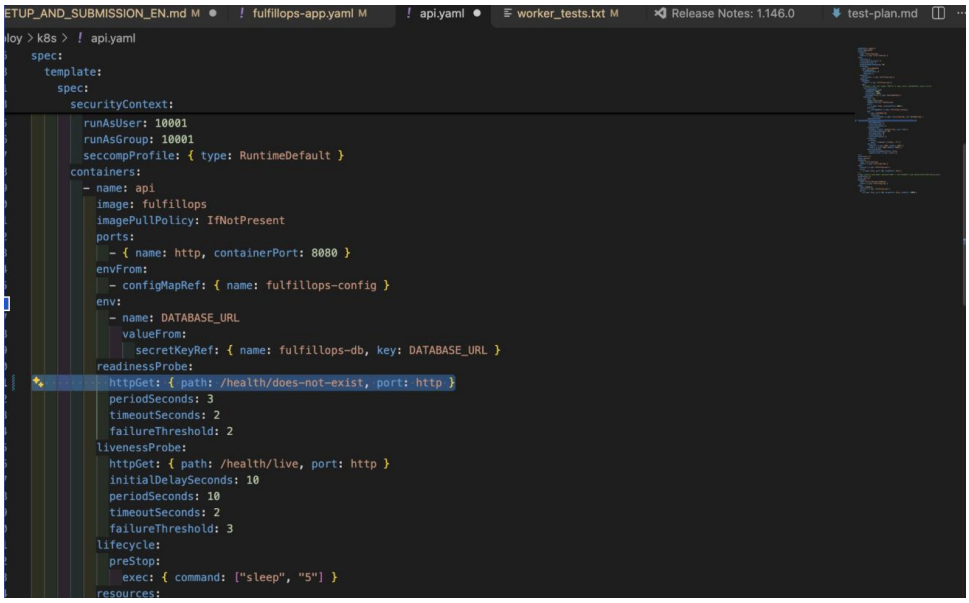


Figure 7: *deploy/k8s/api.yaml with the readiness probe path changed to /health/does-not-exist for the broken-rollout test.*

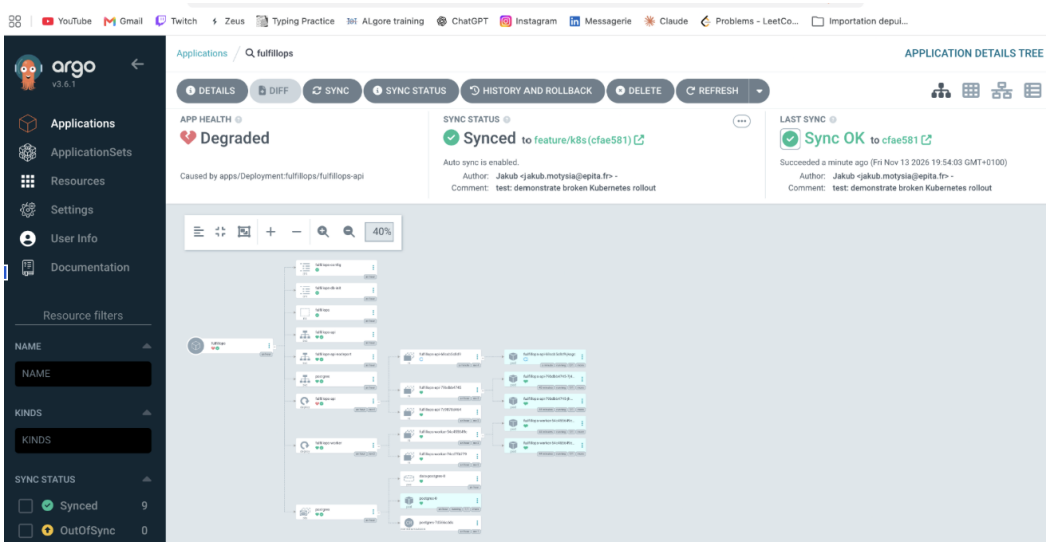


Figure 8: *Argo CD after syncing commit cfae581: application Degraded, caused by deployment fulfillops-api, while the earlier pods stay healthy.*

The recovery half of this test, reverting the bad commit and watching Argo CD restore two ready API pods, is not evidenced in the materials supplied: the revert commit identifier, a post-recovery screenshot and the `kubectl rollout history` output have not been captured. This is listed as remaining work in section 12.

9. Security and operational safeguards

Security is implemented as a set of small controls across the request path, runtime, database, and delivery pipeline:

- request bodies are capped at 16 KiB;
- validation rejects malformed and out-of-range values early;
- errors never expose SQL, credentials, stack traces, or connection strings;
- credentials are supplied through environment variables;
- `.env` is ignored and only placeholders belong in `.env.example`;
- the container runs as a non-root user;
- the reset script refuses production-like environments and database names;
- metrics do not contain customer identifiers or idempotency keys; and
- dependency scanning and SBOM generation are part of CI.

The main threats addressed are:

1. **Overselling under concurrency:** prevented by row locks and transactional stock updates.
2. **Duplicate order creation:** prevented by idempotency locking and a unique database mapping.
3. **Payload substitution under a reused key:** prevented by normalized payload hashing and conflict detection.
4. **Request abuse through oversized or malformed input:** reduced by size limits and strict validation.
5. **Credential leakage:** reduced by environment-based configuration, safe errors, and secret exclusions from Git.
6. **Infinite retry loops:** prevented by maximum attempts and dead-letter state.
7. **Supply-chain vulnerabilities:** detected by the blocking Trivy gate and corrected through dependency and image updates.

These controls provide defense in depth, but they do not replace production identity management, network policy, encrypted secret storage, database backup testing, or centralized audit logging.

10. Observability and incident response

The API exposes Prometheus-format metrics for:

- `fulfillops_orders_created_total`;
- `fulfillops_orders_failed_total`;
- `fulfillops_worker_retries_total`;
- `fulfillops_http_requests_total`; and
- a latency histogram or equivalent measurement for p95 calculation.

Liveness intentionally remains independent of PostgreSQL: it answers whether the process is alive. Readiness checks whether the database and required migrations are available for safe order acceptance. This distinction helps a deployment controller restart or remove an unhealthy instance without mistaking a database outage for a dead process.

The recommended incident workflow is:

1. identify whether the problem is API availability, database readiness, or worker progress;
2. inspect health responses, structured logs, and retry metrics;
3. verify that committed orders and inventory records still exist;
4. restore capacity or allow a replacement worker to reclaim stale work; and
5. document timeline, impact, root cause, recovery, and prevention actions.

The project brief requires two controlled failure demonstrations: killing a worker and stopping an unhealthy API rollout. Both were exercised on the local Kubernetes cluster and are described in section 10.3 and section 8.5. The subsections below record the availability and latency measurements.

10.1 Rolling update availability test

To check that a rolling update does not interrupt service, the readiness endpoint was polled continuously while `kubectl rollout restart deployment/fulfillops-api` replaced the two API pods.

Item	Recorded value
Environment	Local Kubernetes cluster (kind-fulfillops, namespace fulfillops)
Deployment	`fulfillops-api`, 2 replicas
Target endpoint	`http://localhost:8080/health/ready`
Probe window (as recorded)	2026-11-13T19:14:59Z to 19:15:29Z
Interval and duration	every 0.25 s for 30 s
Samples	120 (`docs/rolling_update_probe.csv`, 121 lines including the header)
Result	120 HTTP 200 responses, 0 failed requests (100% availability)
Observed latency	about 2.21 ms to 27.32 ms (head and tail samples)

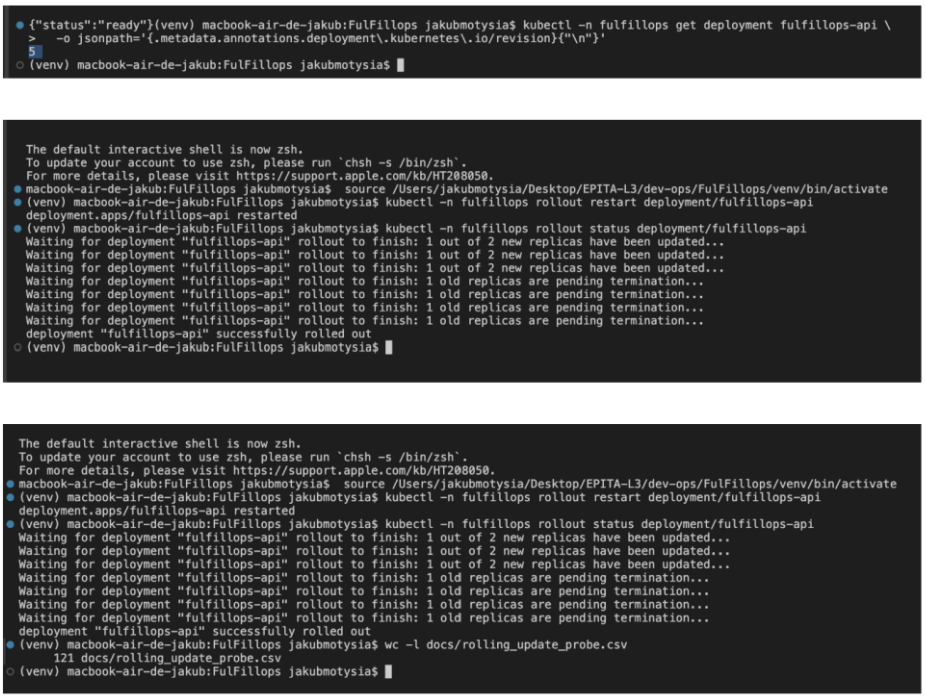


Figure 9: *Terminal output: kubectl rollout restart and rollout status for fulfillops-api (successfully rolled out) and the probe CSV with 121 lines.*

This shows that the readiness endpoint stayed reachable throughout the restart. It does not test order creation, and the probe was run from the same machine as the cluster.

10.2 HTTP sampling and p95 latency

A sampling script (`fulfillops_http_samples.py`) sent 200 sequential requests to `/health/ready` on the local cluster and recorded each latency in `docs/http_samples.csv`.

```
Total samples: 200
Successful HTTP 200: 200
Failed requests: 0
p95 latency: 11.716 ms
```

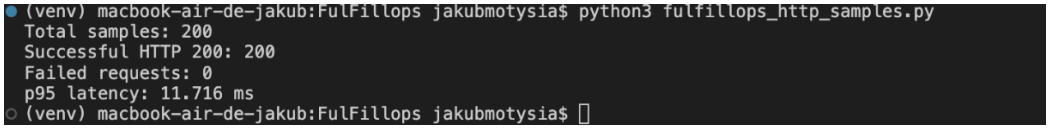


Figure 10: *Output of fulfillops_http_samples.py: 200 samples, 200 HTTP 200, 0 failures, p95 latency 11.716 ms.*

The measurement is local and sequential, without production network latency or concurrent traffic, and it targets a lightweight health endpoint rather than the order-creation path, so it should be read as a baseline, not as a capacity figure.

10.3 Worker pod crash drill (Kubernetes)

A drill script submitted two orders and then force-deleted one of the two worker pods, while checking that the orders were not lost and that Kubernetes restored the desired replica count. The recorded output was:

```
20:15:22Z workers before:
NAME                                  READY   STATUS    RESTARTS   AGE
fulfillops-worker-54c455649c-5clxw   1/1     Running   0          123m
fulfillops-worker-54c455649c-8d5zc   1/1     Running   0          123m
20:15:22Z orders: 9df3b566-3c65-4f7b-869f-9733d29dfa13 de792acd-2bfe-4d78-9585-
bfe010489738
20:15:25Z statuses: completed completed
20:15:25Z killing pod/fulfillops-worker-54c455649c-5clxw mid-processing
pod "fulfillops-worker-54c455649c-5clxw" force deleted from fulfillops namespace
20:15:25Z poll 1: completed completed
20:15:25Z both orders completed after worker crash
NAME                                  READY   STATUS     RESTARTS   AGE
fulfillops-worker-54c455649c-8d5zc   1/1     Running    0          123m
fulfillops-worker-54c455649c-d8md2   0/1     Init:0/1   0          0s
```

The drill shows three things: both committed orders ended as `completed` with no data loss, the surviving worker pod stayed Running, and the Kubernetes ReplicaSet immediately started a replacement pod (`fulfillops-worker-54c455649c-d8md2`) to restore two replicas. The output captured the replacement at Init:0/1, so its transition to Running is not shown.

One limit should be stated plainly. The `statuses` line shows both orders were already `completed` at 20:15:25Z, before the pod was deleted, so the orders were not in flight when the worker was killed. This drill therefore demonstrates replica continuity and self-healing, not recovery of an interrupted job. Recovery of a job abandoned mid-processing is demonstrated by the Docker Compose crash drill described in section 11.2, where one worker was killed with four jobs in flight and all four orders still completed.

11. Team contributions

11.1 Partner 1 (Mohith): application and persistence

Partner 1 implemented the application and persistence foundation:

- FastAPI service and HTTP route behavior;
- PostgreSQL schema, migration, seed data, and configuration;
- SQLAlchemy database models;
- atomic inventory reservation;
- idempotency handling and duplicate SKU aggregation;
- durable worker processing;
- bounded retries and dead-letter handling;
- Docker Compose local runtime integration;
- API, worker, and contract-test support; and
- safe disposable test-database reset tooling.

The contribution was validated through the public checker, the poison-job demonstration, and the documented CI evidence. The main technical ownership was the transaction boundary that connects an accepted order to inventory, idempotency state, and durable work.

11.2 Partner 2 (Jakub): testing, infrastructure, and delivery

Partner 2 implemented and documented the project verification and runtime infrastructure:

- unit, contract, race, worker-state, and worker-reliability tests;
- cross-replica HTTP race tests using `api` and `api2`;
- two-worker claim-safety and abandoned-job recovery tests;
- the worker crash drill and its recorded recovery evidence;
- the deterministic poison-job drill and dead-letter evidence;
- safe test-database reset tooling and test-environment guards;
- Dockerfile and Docker Compose services for PostgreSQL, two API instances, and two workers;
- non-root container execution and health checks;
- the GitHub Actions workflow for linting, tests, contract checks, image building, vulnerability scanning, and SBOM publication; and
- the CI evidence documenting the red Trivy run and corrected green run.

The recorded worker crash drill killed one worker while four jobs were in flight. The surviving worker eventually completed all four orders, and the abandoned job was reclaimed with its attempt count increased to two. The poison-job drill reached `dead_letter` after three attempts, changed the order to `failed`, and kept the reserved stock unreplenished for manual reconciliation.

11.3 Joint ownership

The complete solution depends on both contributions. Partner 1’s transaction and worker implementation provides the behavior being tested; Partner 2’s tests, Compose topology, CI workflow, and evidence demonstrate that behavior under concurrency and failure. Both partners should understand the system end to end because the individual defense can examine any failure path.

12. Limitations and future improvements

The implementation has several limitations that are appropriate to state explicitly:

1. **Some operational evidence is still incomplete.** The Argo CD screenshots were captured while the application tracked the `feature/k8s` branch (commit `c5f0b2e`), not `main`. The revert-and-recover half of the broken-rollout test has not been captured with its commit identifiers, rollout history and screenshot. The scale-based drift test is described in the notes but has no screenshot, and a completed incident postmortem is not included.
2. **Local performance results are environment-specific.** The p95 of 11.716 ms comes from 200 sequential requests to `/health/ready` on a local kind cluster. It does not represent production latency or capacity, and it does not exercise order creation or the database write path.

3. **The Kubernetes worker drill did not interrupt in-flight work.** Both orders had already completed when the pod was deleted. The ConfigMap exposes a `WORKER_PROCESS_DELAY_SECONDS` setting (currently 0) that could be raised so that a pod can be killed while jobs are processing.
4. **Dead-letter jobs require human reconciliation.** Retaining stock is safe against accidental double fulfillment, but an operational procedure is needed to cancel, retry, or manually reconcile a failed order.
5. **The metrics surface is intentionally small.** Production operations would benefit from database pool saturation, queue age, lease-expiry counts, worker throughput, and deployment metadata.
6. **PostgreSQL is a single durable dependency.** High availability, backups, restore drills, and failover would be necessary for a production service.
7. **The poison hook is a test-only mechanism.** It must remain disabled or inaccessible in a production environment.

Future work should first close the remaining evidence gaps: point Argo CD at `main` and recapture the synced state, complete and document the revert-recovery step, and repeat the worker drill with an artificial processing delay. After that, latency should be measured on the order-creation path under concurrent load. The next product-level improvement would be an authenticated operator workflow for dead-letter review and reconciliation.

13. Conclusion

FulfillOps demonstrates the core engineering principle of a reliable order system: correctness must be established at the persistence boundary, not assumed from a successful HTTP response. PostgreSQL transactions and row-level locking protect inventory; durable idempotency mappings protect retries; and the work table allows an independent worker to recover after failure. Bounded retry and dead-letter states make failure visible and finite.

The documented evidence shows successful contract behavior, concurrent reservation handling, worker completion, poison-job termination, a CI security gate that detects and then resolves vulnerable dependencies, and a Kubernetes deployment reconciled by Argo CD with drift detection, a contained broken rollout, a zero-failure rolling restart probe and a worker-pod replacement. The remaining work is mainly completing the evidence: tracking `main`, documenting the revert recovery, interrupting in-flight work in Kubernetes, and measuring performance under realistic load. Clearly separating verified behavior from planned evidence keeps the report credible and provides a precise checklist for final project completion.

14. Reproducibility appendix

Start the local stack

```
docker compose up -d --build
docker compose ps
curl http://localhost:8080/health/ready
```

Run tests

```
python -m pytest
```

Run the contract checker

Run only against a fresh isolated test database:

```
python fulfillops_public_checker.py `
  --base-url http://localhost:8080 `
  --concurrency `
  --stress
```

Reset the disposable test database

```
docker compose exec -T `
  -e APP_ENV=test `
  -e
DATABASE_URL=postgresql+psycopg://fulfillops:fulfillops@postgres:5432/fulfillops
`
  api python scripts/reset_test_data.py
```

Assistance disclosure

Complete this section according to the actual submission policy. Record any AI, framework, documentation, or external code assistance used during development, including what was accepted, reviewed, and changed by the team.